

M14 / PROTECTED ANSWERS

M14 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
DE14-01 Batch vs streaming	3
DE14-02 Events, brokers and topics	4
DE14-03 Producers and message keys	4
DE14-04 Consumers and safe processing	5
DE14-05 Partitions and ordering	6
DE14-06 Offsets, group position and replay	7
DE14-07 Consumer groups and scaling	7
DE14-08 Delivery semantics and idempotency	8
DE14-09 Serialization, schemas and evolution	9
DE14-10 Python producer and consumer workflow	9
DE14-11 Spark Structured Streaming + Kafka	10
DE14-12 Lag, failures and streaming operations	11

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Protected learning support. Open only after a saved genuine attempt. Each solution explains the reasoning and a changed retry.

DE14-01 | Batch vs streaming

Worked recovery: A batch/stream decision should start from business latency and recovery needs, not tool prestige. For example, nightly payroll has a natural business boundary and usually benefits from batch simplicity. Parcel fraud alerts have a low-latency requirement, so events should be available continuously. A hybrid system can stream raw events into durable storage and still compute some official reports in batch.

Why the tempting wrong answer fails: Wrong answer: stream everything because streaming is newer. That ignores business latency, recovery and reconciliation. Batch can be the best fit for bounded official work.

Exact validation evidence: Validation: write latency targets for instant fraud alert, five-minute operations and nightly settlement, then name the replay/recovery source for each.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. An operations dashboard is 25 minutes behind. The producer is healthy and events are durable in Kafka. Is this data loss? Not necessarily. First separate event-time freshness from consumer progress: inspect the newest produced offset/timestamp, the consumer group's committed position and processing rate. Growing lag is a capacity or failure symptom, not proof that events disappeared. Now compare that symptom with the expected invariant: You should be able to classify a workload by bounded/unbounded input, required response time, replay need and failure tolerance. You should also be able to say that Spark Structured Streaming usually executes continuous logic as repeated micro-batches even though the logical input is unbounded. The tempting failure mode is: A common beginner mistake is treating streaming as 'batch every second'. That misses persistent offsets, long-running consumer state, backpressure/lag and restart behavior. Another mistake is selecting Kafka because the source updates frequently even when the business only needs a daily result. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: if the fraud engine must react in 3 seconds but customer analytics may lag 10 minutes, keep one durable stream and let separate consumers process at different rates/group identities.

Fresh changed retry: A supplier uploads one CSV every 20 minutes but inventory alerts are due within 2 minutes of file arrival. Choose the simplest execution model that meets that SLA.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-02 | Events, brokers and topics

Worked recovery: A topic is a retained append-only log. Brokers store topic partitions. Consumers read without deleting for other groups. Therefore a second group can replay retained events. If a new consumer sees only fresh events, first check group/starting position before claiming the broker deleted history.

Why the tempting wrong answer fails: Wrong answer: consumer A read the event so consumer B cannot. Kafka retains records; independent groups track independent progress. Retention is not infinite.

Exact validation evidence: Validation: publish three records, read them with group-A, then with fresh group-B from the beginning. Save topic, partition and offsets from both reads.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A team says yesterday's events are gone because their new consumer starts and sees only fresh traffic. Before blaming retention, check the consumer starting position and group state. A new group with `auto.offset.reset=latest` may begin at the end; that is different from the broker deleting history. Now compare that symptom with the expected invariant: A topic description should show at least one partition and leader. The console consumer should print the produced records. Re-running a consumer with an explicit from-beginning route can read retained records again. The tempting failure mode is: If the client cannot connect, distinguish 'container not running' from 'topic missing' from 'listener/port wrong'. Do not randomly change ports. First prove docker ps shows the broker, then list topics with the broker-local CLI, then test the host port. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: make the topic an audit source. Retention becomes a replay SLA, while group offsets still represent each consumer application progress.

Fresh changed retry: An audit team joins six days later but the topic retains only three days. Explain what Kafka can replay and what raw/archive design would have covered the gap.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-03 | Producers and message keys

Worked recovery: Use the business entity whose order matters as the message key. For order lifecycle events that is usually `order_id`. Delivery callbacks prove broker outcome; an immediate local print does not. Stable keys help keep related events in one partition while partition count is stable.

Why the tempting wrong answer fails: Wrong answer: key by customer when the required order is actually per order. An over-broad key can create hotspots. Also wrong: treating `produce()` return as broker acknowledgement.

Exact validation evidence: Validation: send three O17 events with one key and capture delivery callback topic/partition/offset. Explain the ordering boundary and partition-change caveat.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A customer's status appears as DELIVERED before OUT_FOR_DELIVERY in a downstream service. Before blaming Kafka, ask whether both events had the same key and therefore the same partition. Kafka guarantees order within a partition, not global topic order across partitions. Now compare that symptom with the expected invariant: Each callback should identify a topic/partition/offset or an error. Repeated events with the same key should be routed consistently by a stable partitioner while the partition count is unchanged. Do not promise a partition number across a partition-count change. The tempting failure mode is: Two dangerous beginner habits are ignoring delivery callbacks and using a random UUID as the key for every event. The first hides failed delivery; the second destroys useful grouping. Also do not hard-code a partition unless you own the operational consequences. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: a giant customer creates a hot key. Refine to order_id for order processing, then compute customer aggregates downstream instead of forcing all customer traffic through one partition.

Fresh changed retry: One customer creates 40% of orders. Redesign the key so per-order order is preserved without one customer hotspot.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-04 | Consumers and safe processing

Worked recovery: The safer at-least-once pattern is durable effect first, commit after. That leaves a replay window, so the sink must be idempotent. Commit-first leaves a loss window. External APIs are not automatically covered by Kafka transactions.

Why the tempting wrong answer fails: Wrong answer: commit first, then write the sink. A crash after commit can lose the business effect. Console-only poison-record logging is not durable handling.

Exact validation evidence: Validation: save a valid processed-ledger entry before commit. For malformed input save quarantine JSONL with topic/partition/offset/raw/error, flush/fsync it, then commit under the course policy.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A payment consumer restarted and sent one email twice. The event existed once in Kafka. That can still happen: the email side effect succeeded, the process died before progress was safely recorded, and the event replayed. Fix the side-effect contract or deduplication, not the event log. Now compare that symptom with the expected invariant: After a clean run, the sink contains one durable row per event_id and the group has a committed position beyond processed records. If the process is killed after the sink write but before the commit, replay may happen; the idempotent sink prevents a duplicate business effect. The tempting failure mode is: Committing first is the classic loss window. Writing an external API/database and assuming Kafka transactions cover that external side effect is another mistake. Kafka's own transactional guarantees do not magically make arbitrary third-party systems transactional. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were

silently lost or duplicated.

Changed retry: for a REST API, use a durable idempotency key/outbox or a service-supported Idempotency-Key so replay does not repeat the business action.

Fresh changed retry: Replace the file/database sink with a payment API. Design idempotency/outbox state so a crash after remote success but before commit cannot charge twice.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-05 | Partitions and ordering

Worked recovery: Kafka ordering is partition-local. Four partitions are four ordered logs, not one globally ordered stream. A key that sends one entity to one partition gives per-entity order, but concentrated keys can produce skew.

Why the tempting wrong answer fails: Wrong answer: Kafka guarantees order, without saying partition. Also wrong: add consumers to fix one hot partition.

Exact validation evidence: Validation: print entity, partition and offset for keyed events.

Demonstrate local order while a merged view across partitions can interleave. Count events per partition to expose skew.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. Only partition 2 has rising lag while partitions 0,1,3 are healthy. This is a clue: inspect key distribution and slow records for that partition. Scaling consumers cannot overcome a single hot partition if only one consumer can actively process that partition in the group at a time. Now compare that symptom with the expected invariant: You should be able to reconstruct each business entity's local order from its partition sequence. You should not claim the combined topic has a single meaningful order unless the topic has one partition or the application builds another ordering mechanism. The tempting failure mode is: Increasing partitions after designing 'hash(key) -> partition' can move future records for the same key, depending on the partitioner. Also, choosing customer_id as a key may preserve customer order but create a hotspot for a huge customer. Key design is a business and distribution decision together. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: when one seller dominates, use order_id for order processing and derive seller-level state through aggregation; do not preserve an unnecessarily broad seller-wide total order.

Fresh changed retry: One seller dominates half of orders. Choose a finer processing key while preserving the ability to compute seller totals downstream.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-06 | Offsets, group position and replay

Worked recovery: Offsets are partition-local positions. A committed offset is a group bookmark. Replaying means choosing an earlier retained position; it does not modify the log.

`auto.offset.reset` applies when there is no valid committed position, not as a universal rewind switch.

Why the tempting wrong answer fails: Wrong answer: `auto.offset.reset=earliest` always rewinds an existing group. Valid committed offsets normally determine restart. Offsets are also not timestamps.

Exact validation evidence: Validation: show committed and log-end positions per partition, calculate lag, and specify an exact replay range or group identity. Confirm retention first.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A bug corrupted derived rows between noon and 12:20. You fix the code and want to replay. First identify the source partitions and time/offset boundary, preserve existing evidence, and ensure the sink is idempotent or is rebuilt in an isolated target. Blindly rewinding a production group can duplicate external effects. Now compare that symptom with the expected invariant: Same group + valid commit resumes from its recorded positions. New group behavior depends on reset policy. Replay works only while the required offsets still exist under retention. The tempting failure mode is: `auto.offset.reset` does not normally override an existing valid committed offset. Another common mistake is saying 'offset 50' without naming the partition; offsets are partition-local. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: if retention has already deleted the required offsets, replay must come from another durable raw/archive source. This is why earlier modules preserved raw evidence.

Fresh changed retry: The needed Kafka offsets have expired. Design repair from Bronze/raw archive without double-applying corrected output.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-07 | Consumer groups and scaling

Worked recovery: Consumers sharing group.id share partition work. Different group IDs independently read the topic. With 6 partitions, more than 6 consumers cannot all own one of those partitions at the same moment in the normal group model.

Why the tempting wrong answer fails: Wrong answer: ten consumers create ten-way active parallelism for six partitions. Some consumers will be idle. Different applications that both need all events should not share one group ID.

Exact validation evidence: Validation: draw ownership for 6 partitions with 1, 3, 6 and 8 consumers; then show two independent groups reading the same topic.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A deployment doubles replicas from 3 to 8, but throughput barely changes on a 3-partition topic. That is expected: normal group parallelism is partition-bounded. Either processing per partition must get faster or partitioning must be reconsidered with key/order consequences. Now compare

that symptom with the expected invariant: You should be able to predict whether adding consumers increases parallelism and whether two consumers will share work or each receive all events based on group.id. The tempting failure mode is: Starting every replica with a random group.id creates independent readers and can multiply side effects. Starting unrelated applications with the same group.id makes them accidentally share work. Both bugs look like 'Kafka randomly lost/duplicated messages' until group identity is inspected. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: 24 fraud workers with 12 partitions require either faster per-partition work or more partitions, but repartitioning must respect key/order and hotspot consequences.

Fresh changed retry: Twenty-four fraud workers read twelve partitions but throughput remains low. Decide what evidence would distinguish partition shortage, skew, processing cost and sink bottleneck.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-08 | Delivery semantics and idempotency

Worked recovery: Producer idempotence prevents duplicates caused by retrying the same producer sequence under supported settings. It does not deduplicate two intentional business events. Kafka transactions can atomically coordinate Kafka records/offsets, but an external database/API needs its own idempotency/coordination design.

Why the tempting wrong answer fails: Wrong answer: Kafka idempotence means my payment happens once. Producer idempotence and Kafka transactions have narrower boundaries than arbitrary external business effects.

Exact validation evidence: Validation: feed the same event_id twice to append-only and idempotent sinks. Show two transport rows versus one business effect and name which layer created the outcome.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A consumer writes to a payment database and then commits its offset. It crashes after the DB commit. On restart the event is replayed. A Kafka transaction cannot roll back the external DB commit. Use a business idempotency key, transactional outbox/inbox, or equivalent durable coordination pattern. Now compare that symptom with the expected invariant: You should be able to draw a boundary around what Kafka's idempotent producer/transactions protect and what still requires application or sink idempotency. Current Kafka producer idempotence is enabled by default when no conflicting configs disable it; acks=all is part of the compatible setting set. The tempting failure mode is: Saying 'Kafka guarantees exactly once end to end' without naming source, processor and sink is unsafe. Another mistake is using payload equality as deduplication identity; two legitimate identical-valued events can exist. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: map event_id to the external service idempotency key and save request/result evidence so a replay can safely return the already-applied result.

Fresh changed retry: An email API accepts an Idempotency-Key for only 24 hours. Explain what durable local state you still need for a replay one week later.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-09 | Serialization, schemas and evolution

Worked recovery: A schema is a contract over serialized bytes. Adding an optional field with a compatible default is generally safer than renaming a required field or changing a number into a formatted string. Exact compatibility depends on the schema format and registry policy.

Why the tempting wrong answer fails: Wrong answer: JSON is schema-free. It still has an implicit contract. Also wrong: assuming every additive field is compatible under every schema format and registry policy.

Exact validation evidence: Validation: show a v1 reader accepting an additive optional v2 field and rejecting a deliberately breaking rename/type change. Record actual registry policy when using one.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. A producer changes amount from numeric 19.99 to string '19.99 INR'. Some consumers silently cast; others fail. The real problem is contract evolution without controlled compatibility.

Restore/introduce a schema boundary and change representation through an explicit versioned migration. Now compare that symptom with the expected invariant: You should be able to classify additive optional changes as generally safer than destructive rename/type changes, while acknowledging exact compatibility rules depend on Avro/Protobuf/JSON Schema and registry configuration. The tempting failure mode is: 'JSON is self-describing so we do not need schemas' is a weak production argument. JSON syntax tells you that a field exists, not whether a business contract permits it, whether it is required, or whether changing its type is compatible. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: introduce phones_v2 as an optional array while retaining phone during transition; migrate consumers, then retire the old field through a controlled version change.

Fresh changed retry: Migrate phone:string to phones:[string]. Design a transition where old and new consumers coexist before retirement of the old field.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-10 | Python producer and consumer workflow

Worked recovery: The Python Producer is asynchronous: use delivery callbacks and flush/poll lifecycle correctly. The Consumer needs stable group identity, explicit progress semantics, validation/quarantine and close(). A healthy process is not proof of reconciled data.

Why the tempting wrong answer fails: Wrong answer: the Python process printed SENT, therefore broker delivery is proven. Producer delivery is asynchronous. Invalid consumer records also must not disappear while offsets advance.

Exact validation evidence: Validation: save producer callback partition/offset; on consume save event_id plus partition/offset and durable handled outcome. Malformed input must appear in quarantine before commit under this lesson policy.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. The script prints 'produced' before the callback reports an error. That is why enqueueing locally is not delivery confirmation. Base evidence on callbacks/flush results and broker-side consumption, not a print immediately after produce(). Now compare that symptom with the expected invariant: Producer output should report delivered topic/partition/offset. Consumer output should show decoded key/value plus partition/offset, and a second run of the same group should resume after committed work. A fresh group can replay retained records according to its reset policy. The tempting failure mode is: Calling flush after every single event makes an asynchronous producer behave synchronously and can destroy throughput; never flushing before process exit can leave queued events unsent. On the consumer side, a busy loop with no error handling can hide broker/client problems. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: malformed JSON should create a visible quarantine outcome with topic/partition/offset and raw payload, not infinite crash/retry or silent success.

Fresh changed retry: Kill the consumer after a valid sink write but before commit. Restart and prove the replay causes only one business effect.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-11 | Spark Structured Streaming + Kafka

Worked recovery: Spark Structured Streaming reads Kafka as a source but tracks query progress/checkpoint state using Spark mechanisms. A checkpoint is part of recovery correctness. Deleting it can cause reprocessing or changed state semantics, especially with a non-idempotent sink.

Why the tempting wrong answer fails: Wrong answer: a Spark checkpoint is temporary clutter you delete when stuck. It is recovery state. startingOffsets also does not simply override an established checkpoint on every restart.

Exact validation evidence: Validation: save checkpoint path, Kafka partition/offset metadata and output for a small run. Restart with the same checkpoint and explain the observed resume behavior.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. An engineer deletes the Spark checkpoint to 'fix' a stuck query and the pipeline reprocesses old offsets into a non-idempotent sink. The checkpoint was part of correctness, not disposable cache. Recovery

should diagnose source offsets, query state and sink idempotency before destructive reset. Now compare that symptom with the expected invariant: The streaming DataFrame schema includes Kafka metadata. Valid events appear after parsing. The checkpoint directory is created after the query starts. Restart behavior follows checkpoint state when it exists. The tempting failure mode is: Do not confuse a Kafka consumer group's committed offsets with Spark Structured Streaming checkpoint semantics. Spark's Kafka source manages its query progress through Spark checkpointing; the integration docs explicitly describe Kafka-specific configuration boundaries. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: route invalid records to quarantine and reconcile input count = valid + quarantine for a bounded observation window/micro-batch evidence.

Fresh changed retry: Start a corrected query with a new checkpoint and isolated sink for a bounded replay. State reconciliation required before promotion.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.

DE14-12 | Lag, failures and streaming operations

Worked recovery: Consumer lag is per-partition distance between produced/log-end progress and consumer progress. Growing lag means the application is falling behind, but the cause could be skew, slow sink, poison records, insufficient capacity or rebalance churn.

Why the tempting wrong answer fails: Wrong answer: lag is high, so add consumers. That can fail for one hot partition, a saturated sink or rebalance churn. Process health alone is not correctness proof.

Exact validation evidence: Validation: record log-end, committed position and lag per partition over time together with assignment/rebalance logs and processing/sink latency. Require output reconciliation too.

Recovery walkthrough: Start from the observed incident, not from a favorite Kafka setting. Lag spikes after a deployment. Errors are low but rebalances occur every minute. Check whether consumers are missing heartbeat/poll requirements because processing blocks too long. A scaling change can make things worse if it increases rebalance churn without adding useful partition parallelism. Now compare that symptom with the expected invariant: You should be able to read a per-partition lag table, identify whether lag is uniform or skewed, and choose the next evidence to collect: input rate, processing latency, error rate, rebalance logs, partition/key distribution and sink health. The tempting failure mode is: Lag=0 is not automatically healthy if no events are arriving, and nonzero lag is not automatically an incident. Alerts need business freshness SLOs and trend/rate context. Another failure is sending poison records to a DLQ and never operating the DLQ. A complete recovery note should name the boundary that failed, the evidence that supports the hypothesis, the smallest safe correction, and the reconciliation that proves no records/business effects were silently lost or duplicated.

Changed retry: if only one partition lags, investigate its keys/records and processing latency before scaling the whole cluster; a hot partition often will not be fixed by idle extra consumers.

Fresh changed retry: After a deployment all partitions lag and rebalances spike. Build an evidence order starting with deployment, processing, polling and sink health rather than random restarts.

Teach-back proof: explain the boundary/guarantee in two sentences without using the review wording, then name the evidence that could prove you wrong.